



Une entreprise ordinaire à l'agilité extraordinaire

FORMATION JAVA SPRING ANGULAR

ANGULAR AVANCE

mohamed.el-babili@fms-ea.com

33+ 628 111 476

Version : 3.0

DMAJ : 04/03/24

Module : DEV-ANGU-002

Angular avancé

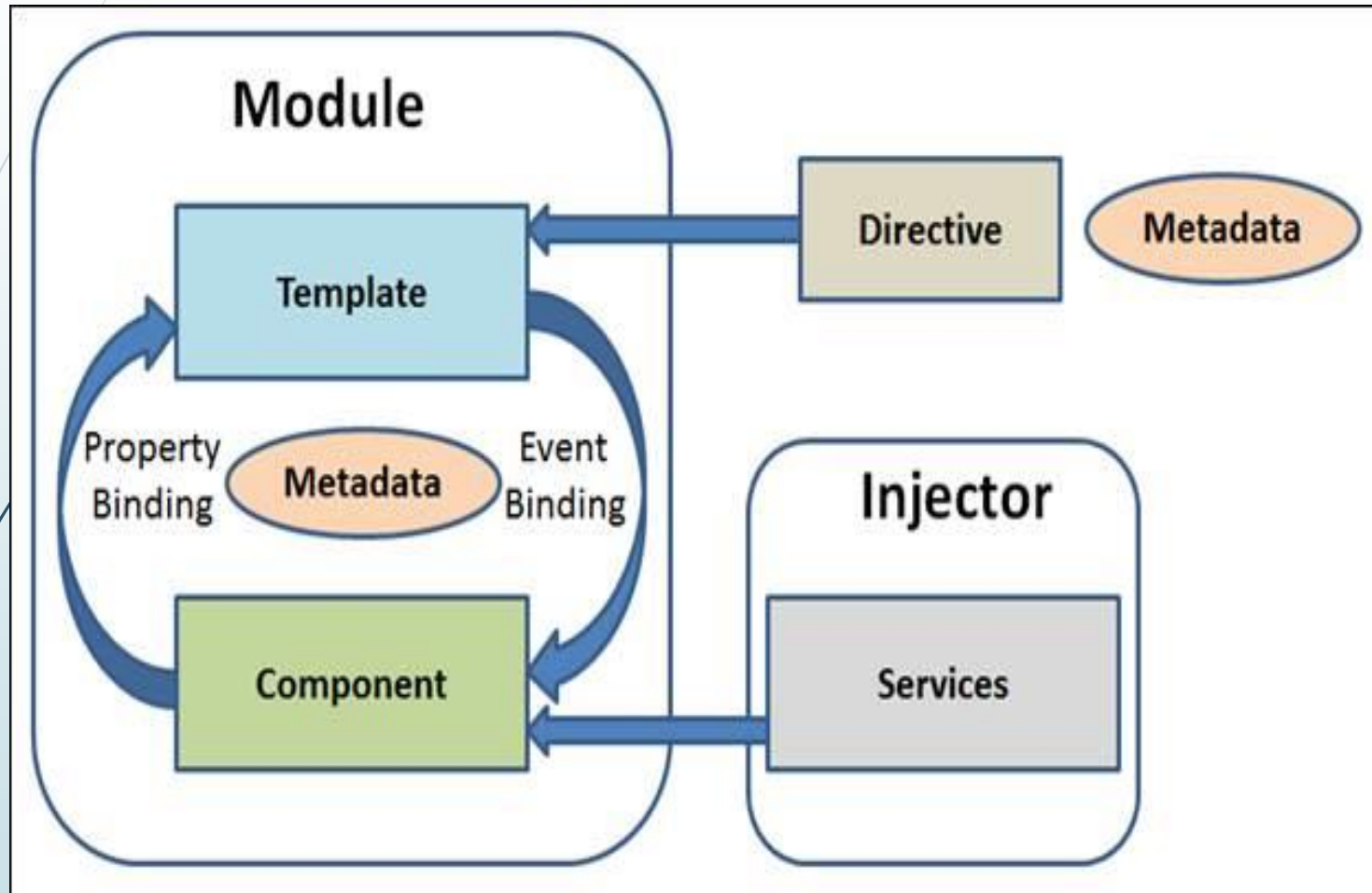
Angular	01 Connaître et Utiliser Typescript
	02 Connaître l'historique et l'architecture d'Angular
	03 Réalisation d'une SPA (module, composant, service, routage, com...)
Angular avancé	01 Utiliser les concepts avancés : RXJS, Hooks, Pipes, ReactiveForms/ FormBuilder, Guards
Angular & NgRx	01 Utiliser NgRx dans une application Angular (State, Action,



ANGULAR avancé

- Synthèse
- MVC & MVVM
- Protocole http et Api Rest
- Mise en œuvre de Json Server
- Application Node step by step
- Api Rest & Bdd
- Utilisation d'un client Rest
- Programmation réactive
- Cycle de vie d'un composant & hooks
- Pipes
- Différence entre Ngcontainer & ngtemplate
- FormsModule, ReactiveFoms & FormsBuilder
- Sécurisé une appli avec les Guards
- Les modules
- Aller encore + loin
- Ressources

Synthèse sur ce qui a été vu



MVC & MVVM

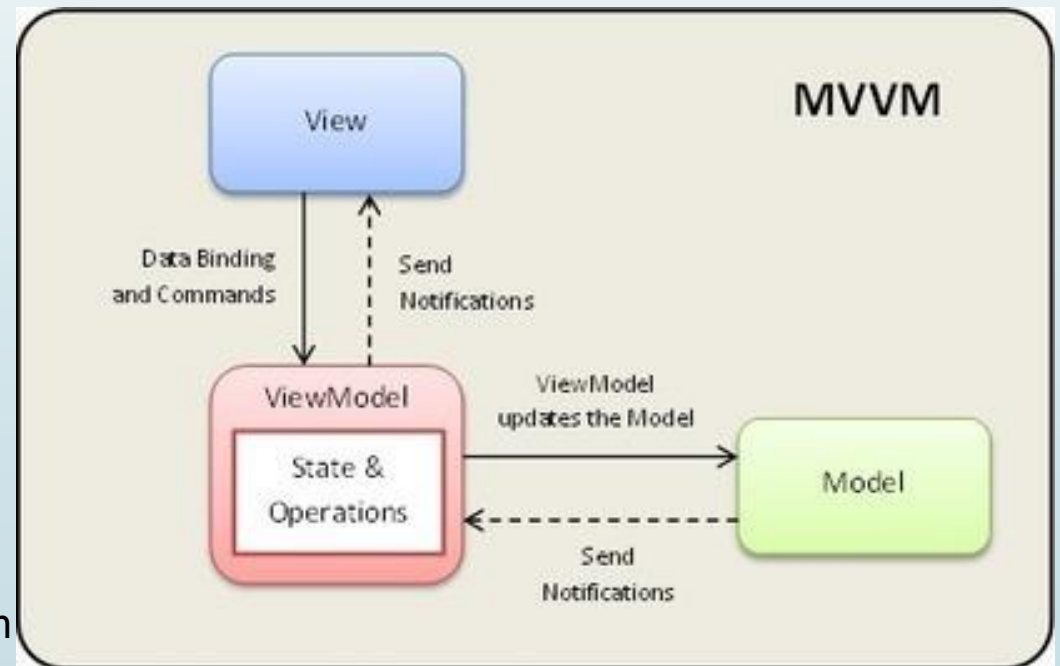
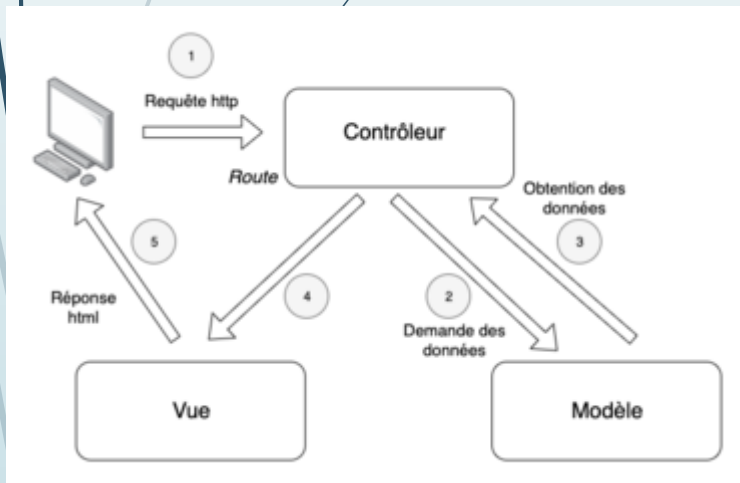
Angular est un framework qui facilite le développement d'applications web évolutives, robustes et maintenables. Sauf que + l'application est complexe, + c'est difficile de maintenir, gérer ou organiser le code. Il est possible d'utiliser une architecture MVC mais le MVVM est plus approprié avec Angular.

L'architecture MVVM (Model-View-ViewModel) est une approche de conception logicielle qui vise à séparer les données d'une application (le modèle) de sa présentation (la vue).

Le ViewModel est une sorte de pont entre le modèle et la vue, il fournit une interface que la vue peut utiliser pour accéder aux données du modèle de manière appropriée.

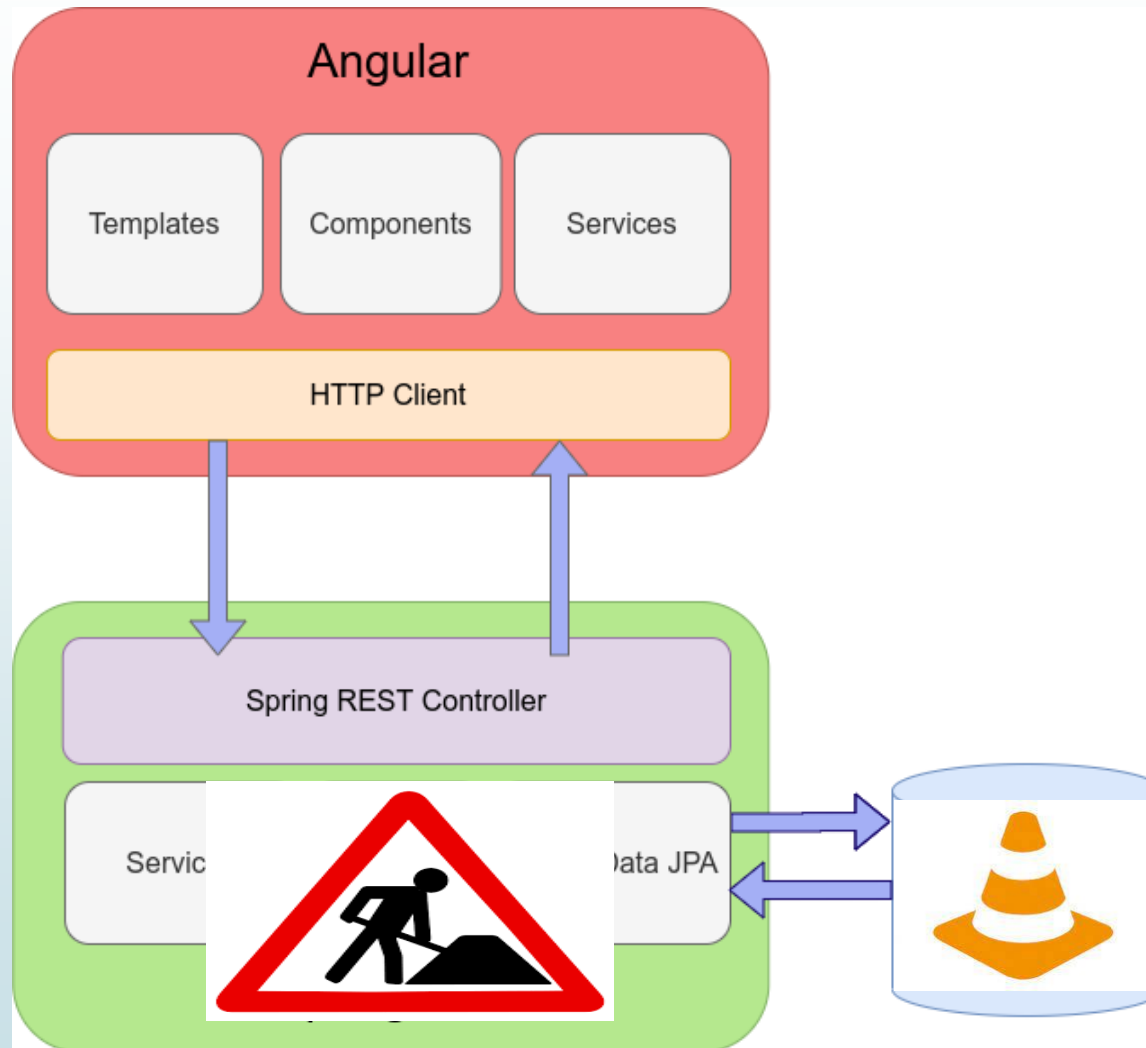
Cette architecture permet de rendre les vues plus indépendantes du modèle, ce qui facilite la maintenance et l'évolution de l'application.

MVC



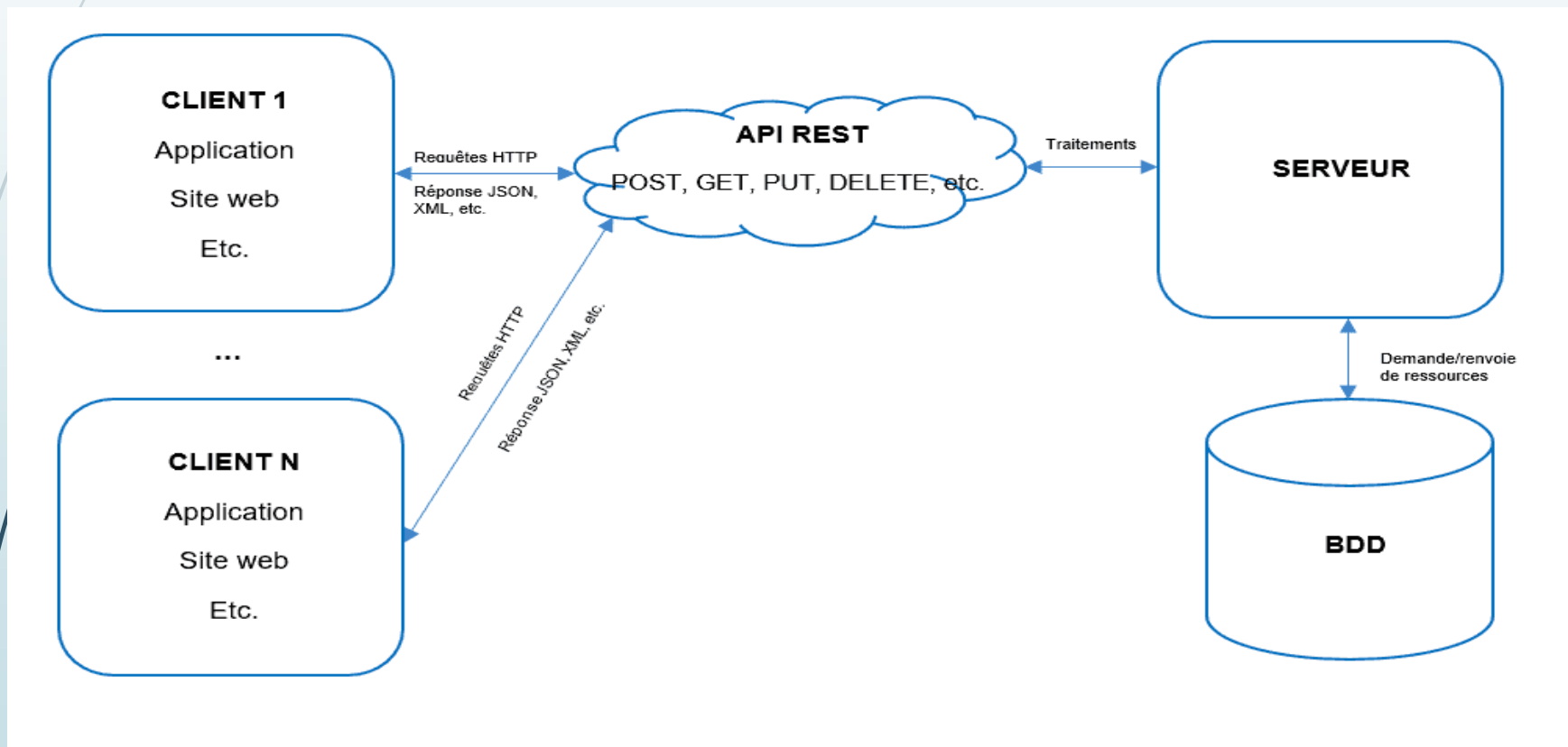
Dans le choix d'une architecture, il faut bien mesurer les avantages et inconvénients

C'est bien beau tout ça mais comment je vais simuler l'activité finale de mon application quand l'appli/api côté back n'est pas prête !



Protocole Http & Api Rest

- L'Hypertext Transfert Protocol(http) est un protocole de transfert hypertexte, de communication client serveur développé pour le World Wide Web. Html définit comment un site est construit, http détermine comment la page est transmise du serveur au client.
- Une Api est une interface de programmation d'application (Web ou pas), c'est un contrat entre un utilisateur(appel ou requête) et un fournisseur d'informations(réponse). S'il s'agit d'une Api Rest, elle respecte les contraintes d'architecture REST (Representational State Transfer) et donc permet d'interagir avec les services web RESTfull.



- [Comparatif Api Rest, Soap, GraphQL et Rpc](#)

Mise en œuvre de Json Server

- Comment mettre en œuvre une api en local pour tester une application sans attendre que le back ou la vraie Api soit opérationnelle, bref ne pas rester bloqué : **Json Server** !
 - `npm install json-server@0.17.0 concurrently`
 - Modifier fichier "Package.json"
 - `"start": "concurrently \"ng serve\" \"json-server --watch db.json\"",`
 - Permet de lancer en parallèle les 2 serveurs
- Ajouter une base de donnée "db.json" à la racine du projet
- Tester : `npm start`

```
"dependencies": {  
  "@angular/animations": "~13.0.0",  
  "@angular/common": "~13.0.0",  
  "@angular/compiler": "~13.0.0",  
  "@angular/core": "~13.0.0",  
  "@angular/forms": "~13.0.0",  
  "@angular/platform-browser": "~13.0.0",  
  "@angular/platform-browser-dynamic": "~13.0.0",  
  "@angular/router": "~13.0.0",  
  "@fortawesome/angular-fontawesome": "^0.10.2",  
  "@fortawesome/fontawesome-svg-core": "^6.1.0",  
  "@fortawesome/free-solid-svg-icons": "^6.1.0",  
  "bootstrap": "^5.1.3",  
  "bootstrap-icons": "^1.8.3",  
  "concurrently": "^7.2.0",  
  "font-awesome": "^4.7.0",  
  "json-server": "^0.17.0",  
  "rxjs": "~7.4.0",  
  "tslib": "^2.3.0",  
  "zone.js": "~0.11.4"  
}
```

La aussi, il faut être vigilant sur la compatibilité des versions utilisées.

Application Node step by step

Exécution d'une application Node.js (par ex Angular) étape par étape :

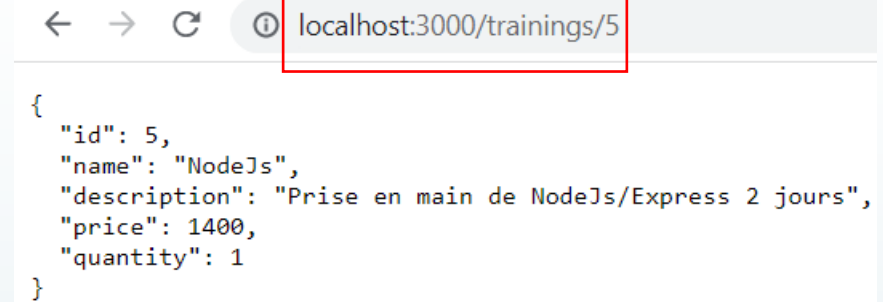
```
package.json > ...
You, 21 months ago | 1 author (You)
1  {
2    "name": "trainings-front-app",
3    "version": "0.0.0",
4    "scripts": {
5      "ng": "ng",
6      "start": "concurrently \"ng serve\" \"\\\"package.json\\\"\"",
7      "build": "ng build",
8      "watch": "ng build --watch --configuration development",
9      "test": "ng test"
10   },
11   "private": true,
12   "dependencies": {
13     "@angular/animations": "~13.0.0",
14     "@angular/common": "~13.0.0",
15     "@angular/compiler": "~13.0.0",
16     "@angular/core": "~13.0.0",
17     "@angular/forms": "~13.0.0",
18     "@angular/platform-browser": "~13.0.0",
19     "@angular/platform-browser-dynamic": "~13.0.0",
20     "@angular/router": "~13.0.0",
21     "@fortawesome/angular-fontawesome": "^0.10.1",
22     "@fortawesome/fontawesome-svg-core": "^6.1.1",
23     "@fortawesome/free-solid-svg-icons": "^6.1.1",
24     "bootstrap": "^5.1.3",
25     "bootstrap-icons": "^1.8.3",
26     "concurrently": "^7.2.0",
27     "font-awesome": "^4.7.0",
28     "json-server": "^0.17.0",
29     "rxjs": "~7.4.0",
30     "tslib": "^2.3.0",
31     "zone.js": "~0.11.4"
32   },
33   "devDependencies": {
34     "@angular-devkit/build-angular": "^13.3.5",
```

- Une application Angular est un projet Node.js avec un fichier « package.json » à sa racine. Ce fichier contient généralement les dépendances du projet nécessaires au fonctionnement de l'appli et les dépendances nécessaires uniquement pendant le processus de développement de l'appli. On peut trouver d'autres éléments toujours sous la forme de clé/valeur telles que le nom ou la version de l'application, ainsi que des scripts servant à lancer des actions plus spécifiques que la simple commande **npm run**
- Lorsque vous exécutez la commande **npm install** après avoir cloné un dépôt Github par ex, Node.js télécharge et installe toutes les dépendances spécifiées dans le fichier « package.json ». Ces dépendances sont généralement stockées dans le répertoire « node_modules » à la racine de votre projet.
- Lorsque Node.js exécute votre application, il résout les dépendances en recherchant d'abord les modules dans le répertoire « node_modules » de votre projet. Si une dépendance n'est pas trouvée localement, Node.js remonte dans l'arborescence des répertoires pour chercher la dépendance dans les répertoires parents. Ce processus continue jusqu'à ce que Node.js trouve la dépendance ou qu'il atteigne le répertoire racine du système de fichiers sans la trouver. En effet, en plus des dépendances locales installées dans le répertoire « node_modules », Node.js peut également utiliser des dépendances installées globalement sur le système. Les dépendances globales sont celles qui sont installées avec l'option **-g** lors de l'utilisation de **npm install**.
- Une fois que toutes les dépendances ont été résolues, Node.js exécute l'application Angular en utilisant ces dépendances pour charger les modules et l'application.

Api Rest & Bdd



```
[
  {
    "id": 1,
    "name": "Java",
    "description": "Formation Java SE 8 sur 5 jours",
    "price": 1500,
    "quantity": 1
  },
  {
    "id": 2,
    "name": "DotNet",
    "description": "Formation DotNet 3 jours",
    "price": 1000,
    "quantity": 1
  },
  {
    "id": 3,
    "name": "Python",
    "description": "Formation Python/Django 5 jours",
    "price": 1300,
    "quantity": 1
  },
  {
    "id": 4,
    "name": "Php",
    "description": "Initiation au Dev/Web avec hp 4 jours",
    "price": 1300,
    "quantity": 1
  },
  {
    "id": 5,
    "name": "NodeJs",
    "description": "Prise en main de NodeJs/Express 2 jours",
    "price": 1400,
    "quantity": 1
  },
  {
    "id": 7,
    "name": "Java Avancé",
    "description": "Formation Java Avancé/Bdd 3 jours",
    "price": 1200,
    "quantity": 1
  }
]
```



```
{
  "id": 5,
  "name": "NodeJs",
  "description": "Prise en main de NodeJs/Express 2 jours",
  "price": 1400,
  "quantity": 1
}
```

Comment faire pour ajouter des formations, supprimer ou mettre à jour ?

Db.json



```
{
  "trainings": [
    {
      "id": 3,
      "name": "Python",
      "description": "Formation Python/Django 10 jours",
      "price": "2800",
      "quantity": 1
    },
    {
      "id": 4,
      "name": "Php",
      "description": "Initiation au Dev/Web avec hp 4 jours",
      "price": 1300,
      "quantity": 1
    },
    {
      "id": 5,
      "name": "NodeJs",
      "description": "Prise en main de NodeJs/Express 2 jours",
      "price": 1400,
      "quantity": 1
    }
  ]
}
```

Utilisation d'un client Rest : ARC (Postman, Insomnia...)

Accueil > Applications > Advanced REST client



Advanced REST client

advancedrestclient.com

★★★★★ 11916 | Extensions |

Method Request URL
GET http://localhost:3000/trainings

Parameters

200 OK 2099.20 ms



```
[Array[6]]
-0: {
  "id": 1,
  "name": "Java",
  "description": "Formation Java SE 8 sur 5 jours",
  "price": 1500,
  "quantity": 1
},
-1: {
  "id": 2,
  "name": "DotNet",
  "description": "Formation DotNet 3 jours",
  "price": 1000,
  "quantity": 1
},
-2: {
  "id": 3,
  "name": "Python",
  "description": "Formation Python/Django 5 jours",
  "price": 1300,
  "quantity": 1
},
-3: {
  "id": 4,
  "name": "Php",
  "description": "Initiation au Dev/Web avec hp 4 jours",
  "price": 1300,
  "quantity": 1
}
```

Request

2

Method Request URL
GET http://localhost:3000/trainings/7

Parameters

200 OK 2068.90 ms



```
{
  "id": 7,
  "name": "Java Avancé",
  "description": "Formation Java Avancé/Bdd 3 jours",
  "price": 1200,
  "quantity": 1
}
```

Method Request URL
DELETE http://localhost:3000/trainings/7

3

Parameters

200 OK 2085.50 ms



```
{}
```

Method Request URL
POST http://localhost:3000/trainings/

4

Parameters

Headers

Body

Variables

Body content type Editor view
application/json Raw input

FORMAT JSON MINIFY JSON

```
{
  "id": 10,
  "name": "PowerBi",
  "description": "Business Intelligence 10 jours",
  "price": 5000,
  "quantity": 1
}
```

201 Created 2082.40 ms



```
{
  "id": 10,
  "name": "PowerBi",
  "description": "Business Intelligence 10 jours",
  "price": 5000,
  "quantity": 1
}
```

Method Request URL
PUT http://localhost:3000/trainings/10

5

Parameters

Headers

Body content type Editor view
application/json Raw input

FORMAT JSON MINIFY JSON

```
{
  "id": 10,
  "name": "PowerBi",
  "description": "Business Intelligence 10 jours",
  "price": 3000,
  "quantity": 1
}
```

200 OK 2058.10 ms



```
{
  "id": 10,
  "name": "PowerBi",
  "description": "Business Intelligence 10 jours",
  "price": 3000,
  "quantity": 1
}
```

6

```
{
  "id": 5,
  "name": "NodeJs",
  "description": "Prise en main de NodeJs/Express 2 jours",
  "price": 1400,
  "quantity": 1
},
{
  "id": 10,
  "name": "PowerBi",
  "description": "Business Intelligence 10 jours",
  "price": 3000,
  "quantity": 1
}
]
```

```
[1] POST /trainings/10 404 24.107 ms - 2
[1] GET /trainings/ 200 31.646 ms - 679
[1] POST /trainings/ 201 26.099 ms - 120
[1] PUT /trainings/ 404 23.935 ms - 2
[1] PUT /trainings/10 200 28.353 ms - 120
[1] GET /trainings/ 200 20.949 ms - 815
```

Programmation réactive

Il est question maintenant d'exploiter notre api dans notre application

```
getAllTrainings() {  
  this.apiService.getTrainings().subscribe({  
    next : (data) => this.listTrainings = data,  
    error : (err) => this.error = err.message,  
    complete : () => this.error = null  
  })  
}
```

Ajouter un service dédié

```
export class ApiService {  
  
  constructor(private http:HttpClient) { }  
  
  public getTrainings() {  
    return this.http.get<Training[]>(environment.host+"/trainings");  
  }  
  
  public getTraining(id : number) {  
    return this.http.get<Training>(environment.host+"/trainings/"+id);  
  }  
}
```

Environnement.ts

```
export const environment = {  
  production: false,  
  host : "http://localhost:3000"  
};
```

Ajouter le module : `HttpClientModule`
Dans le module principal

Cycle de vie d'un composant & Hooks

Un composant passe par différentes étapes de sa naissance à sa mort. Il est possible d'intercepter ces étapes et d'y associer du code via des méthodes prédéfinies (et déclarées dans des interfaces de programmation).

Ces méthodes sont appelées des hooks !

Voici la liste des 4 hooks les plus importants :

1 • **constructor**

2 • **ngOnChanges**

3 • **ngOnInit**

4 • **ngDoCheck**
• **ngAfterContentInit**
• **ngAfterContentChecked**
• **ngAfterViewInit**
• **ngAfterViewChecked**

5 • **ngOnDestroy**

• `ngOnChanges()` : cette méthode est appelée à chaque changement des propriétés de l'objet d'entrée du composant, déclarées par l'annotation `@Input()`.

• `ngOnInit()` : cette méthode est appelée lors de la création du composant.

• `ngDoCheck()` : cette méthode est appelée à chaque itération de la boucle de vérification de l'état du composant, et avant que le change detector ne détermine si des modifications sont apportées aux propriétés du composant

• `ngOnDestroy()` : cette méthode est appelée juste avant que le composant ne soit désalloué.

Un moyen d'observer les hooks

```
export class OrderComponent implements OnInit, OnChanges, DoCheck, OnDestroy {
  dateOrder : Date = new Date();
  constructor(public cartService : CartService, private router : Router) {}

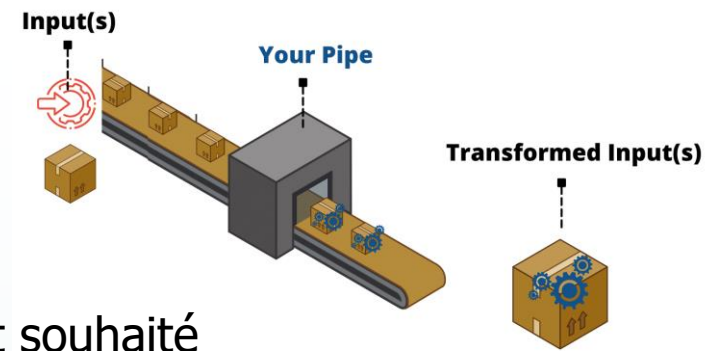
  ngOnChanges(changes: SimpleChanges): void {
    console.log('ngOnChanges' + changes);
  }

  ngOnInit(): void {
    console.log('ngOnInit')
  }

  ngDoCheck(): void {
    console.log('ngDoCheck')
  }

  ngOnDestroy(): void {
    console.log('ngOnDestroy')
  }
}
```

Pipes



Servent à mettre en forme des données dans le format souhaité
Pour ce faire, il y a l'opérateur |

Nous pouvons appliquer des pipes à n'importe quelle vue ou modèle et à toute donnée d'entrée.

Voici quelques exemples d'utilisation :

```
<tr *ngFor="let p of cartService.getCart()">
  <td>{{p.id}}</td>
  <td>{{p.name}}</td>
  <td>{{p.quantity}}</td>
  <td>{{p.price | number:'0.2'}}</td>
</tr>
```

```
<td>{{cartService.getAmount() | currency : 'EUR'}}</td>
```

```
<div class="form-group">
  <label>Date de la commande : </label>
  <label> {{dateOrder | date:"MM/dd/yy"}}</label>
</div>
```

ID	Nom	Quantité	Prix
1	Java	1	1,500.00
2	DotNet	1	1,000.00
Total :			€2,500.00

Date de la commande :05/23/22

ngContainer et ngTemplate

```
<div class="container">
  <ng-container *ngIf="listTrainings else loading">
    <table class="table table-striped">
      <thead>
        <tr>
          <th>ID</th> <th>NOM</th> <th>DESCRIPTION</th> <th>PRIX</th> <th>QUANTITE</th> <th>PANIER</th>
        </tr>
      </thead>
      <tbody>
        <tr *ngFor="let training of listTrainings">
          <td>{{training.id}}</td>
          <td>{{training.name}}</td>
          <td>{{training.description}}</td>
          <td>{{training.price}}</td>
          <td> <input type="number" name="quantity" style="width: 60px" [(ngModel)]="training.quantity">
          <td> <a class="btn btn-primary" (click)="onAddToCart(training)"> Ajouter </a></td>
        </tr>
      </tbody>
    </table>
  </ng-container>
  <ng-template #loading>
    <div>Loading...</div>
  </ng-template>
</div>
```

ng-container sert à regrouper des éléments sans ajouter de nœuds supplémentaires au DOM, il est souvent utilisé avec des directives structurales ***ngIf...**

ng-template permet de créer un contenu dynamique qui n'est pas ajouté au DOM tant que vous ne l'avez pas spécifiquement sollicité, le code peut être appelé plusieurs fois dans l'exemple à l'aide de **loading**

En clair ici, il s'agit d'optimisation de gestion de l'utilisation du DOM

Voir aussi **ng-content**

Template Driven Form Vs Reactive Forms

Jusqu'ici nous avons utilisé les formulaires basés sur un modèle (**FormsModule**), plutôt pratique pour des formulaires simples d'authentification, de contacts ou autre. En bref, ils sont parfaitement **adaptés à l'utilisation de cas simples**.

Les formulaires réactifs (**ReactiveFormsModule**) offrent des **formulaires dynamiques** dont les valeurs changent au fil du temps. Ils peuvent paraître complexe dans leur mise en œuvre mais s'avère très puissants et flexibles. Compte tenu de leur approche, toute la déclaration et la structuration des formulaires est à faire dans le code Typescript exploité côté Html.

NB : Ajouter

ReactiveFormsModule

Dans les imports de app.module

```
import { FormControl, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-customer',
  templateUrl: './customer.component.html',
  styleUrls: ['./customer.component.css']
})
export class CustomerComponent implements OnInit {
  myForm : FormGroup;
  constructor(public cartService : CartService, private router : Router) {
    let customer = this.cartService.getCustomer();
    this.myForm = new FormGroup({
      name : new FormControl(customer.name),
      firstName : new FormControl(customer.firstName),
      address : new FormControl(customer.address),
      phone : new FormControl(customer.phone),
      email : new FormControl(customer.email)
    })
  }

  ngOnInit(): void {
  }

  onSaveCustomer(form : FormGroup){
    if(form.valid){
      this.cartService.saveCustomer(new Customer(form.value.name, form.value.firstName,
        form.value.address, form.value.phone, form.value.email));
      this.router.navigateByUrl('order');
    }
  }
}
```

```
<form [formGroup]="myForm" (ngSubmit)="onSaveCustomer(myForm)">
  <div class="form-group">
    <label class="control-label">Nom : </label>
    <input type="text" formControlName="name" class="form-control">
  </div>
```

Variante avec un FormBuilder

```
ngOnInit(): void {
  this.myForm = this.formBuilder.group({
    name : [this.customer.name, Validators.required],
    firstName : [this.customer.firstName, Validators.required],
    address : [this.customer.address, [Validators.required, Validators.minLength(25)]],
    phone : [this.customer.phone, [Validators.required, Validators.maxLength(10)]],
    email : [this.customer.email, [Validators.required, Validators.pattern('[a-z0-9.@]*')]]
  })
}
```

Sécuriser une appli avec les Guards



Il est parfois nécessaire d'empêcher l'accès vers une route ou composant de votre application pour des raisons de sécurité.

C'est possible à l'aide des guards ou gardes.

Par exemple, un composant d'administration que seules les admins peuvent avoir accès servant à créer des utilisateurs.

```
PS C:\Users\EI-BabiliM\angular-workspace\trainings-front-app-auth> ng g g components/admin
? Which interfaces would you like to implement? (Press <space> to select, <a> to toggle all, <i> to invert selection, and <enter> to proceed)
>(*) CanActivate
  ( ) CanActivateChild
  ( ) CanDeactivate
  ( ) CanLoad
```

```
@Injectable({
  providedIn: 'root'
})
export class AdminGuard implements CanActivate {
  constructor(private authService: AuthService, private router : Router){}

  canActivate(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot): Observable<boolean | UrlTree> | Promise<boolean | UrlTree> | boolean | UrlTree {

    if(this.authService.isAdmin()){
      return true;
    }
    else {
      this.router.navigateByUrl('/login');
      return false;
    }
  }
}
```

App-routing.module.ts

```
{
  path : 'admin' , component : AdminComponent,
  canActivate : [AdminGuard]
},
{ path : '' , redirectTo : 'trainings', pathMatch : 'full' },
{ path: '404', component: NotFoundComponent},
{ path: '**', redirectTo: '/404' }
];
```

App.component.html

```
<li *ngIf="this.authService.isAdmin()" class="nav-item">
  <a class="nav-link" href="#" routerLink="/admin">Administration</a>
</li>
```

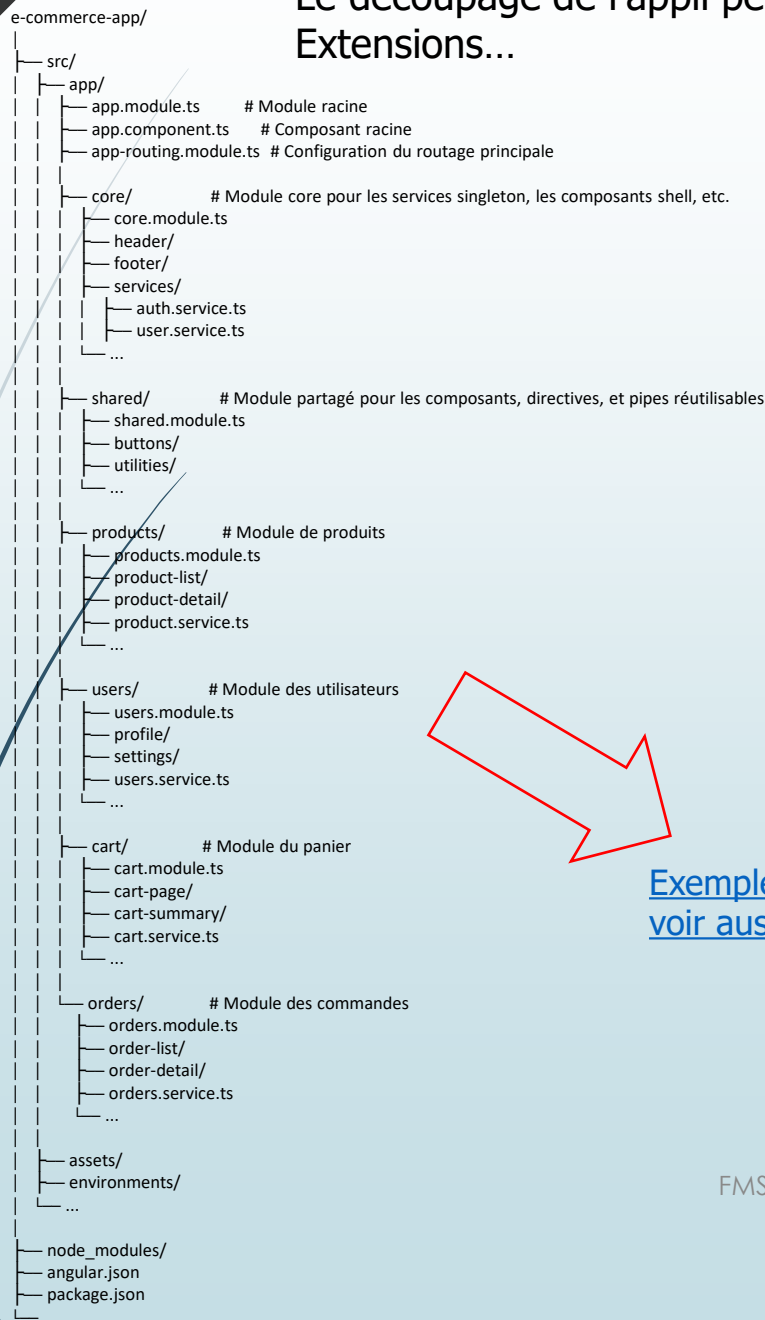
Du coup, grâce à la directive ici, le lien n'apparaît que pour les admins. Et, si qq tente d'accéder à la route correspondante, il est renvoyé vers le formulaire d'authentification

Les modules

Dans une application volumineuse, il est indispensable de la diviser en modules correspondant à un ensemble de fonctionnalités distinctes. Chaque module contenant ses propres composants, directives et services.

Le découpage de l'appli permet : Réutilisation, Lazy Loading, Organisation, Extensions...

Comment générer un module dans une application :



EXPLORER

TEST-NG-APP

- > node_modules
- > src
 - > app
 - > authenticate-module
- > assets
- > environments
- ★ favicon.ico
- < index.html
- TS main.ts

TS authenticate-module.module.ts U X

```
src > app > authenticate-module > TS authenticate-module.module.ts > ...
1 import { NgModule } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3
4 @NgModule({
5   declarations: [],
6   imports: [
7     CommonModule
8   ]
9 })
10 export class AuthenticateModuleModule { }
11
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

● PS C:\Users\EI-Babili\angular-workspace\test-ng-app> ng g m authenticate-module
CREATE src/app/authenticate-module/authenticate-module.module.ts (204 bytes)
○ PS C:\Users\EI-Babili\angular-workspace\test-ng-app>

Exemple de découpage d'une application modulaire, voir aussi les bonnes pratiques associées dites LIFT ...

Aller encore + loin

Exception (Typescript)

[JSDoc](#) Reference (Typescript comments)

@Input / @Output ...

RxJs

NgRx

Test unitaires

.....

AngularMaterial

.....

Build Angular app : [ng build](#)

```
try {  
  throw Error("Oh no");  
}  
catch(e) {  
  console.log(e);  
}
```

```
/**  
 * Méthode qui ajoute une formation au panier  
 * @param training formation à ajouter  
 */  
addTraining(training: Training) {
```

Les Ressources

- <https://angular.io/>
- <https://angular.fr/routing/guard.html>
- <https://guide-angular.wishtack.io/angular/observables/subscribe>
- <https://www.youtube.com/@CoulissesTech>
- <https://deepakjosecodes.com/mvvm-architecture-in-angular-guide-with-code-examples/>
- <https://rxjs.fr/>
- <https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html>
- <https://basarat.gitbook.io/typescript/type-system/exceptions>
- <https://angular.io/guide/deployment>